Visvesvaraya Technological University

Jnana Sangama, Belagavi – 590014

METHODOLOGY — PROJECT WORK PHASE-2

Bluetooth Low Energy and Surveillance Camera-based Navigation System for Visually Impaired People

Submitted in fulfillment of the requirements for the award of the degree of
Bachelor of Engineering in Computer Science and Engineering

SUBMITTED BY

Ms. Aishwarya Kumar Telasinge	2AG23CS007
Ms. Anuja Sutar	2AG23CS016
Ms. Aishwarya Channashetti	2AG23CS006
Ms. Chandrika Bannimatti	2AG23CS032

UNDER THE GUIDANCE OF

Prof. Avanti Patil
Assistant Professor, Dept. of CSE, AITM, Belagavi

Angadi Institute of Technology & Management

Department of Computer Science and Engineering, Belagavi – 590009

2025 – 2026

Methodology

Bluetooth Low Energy and Surveillance Camera-based Navigation System for Visually Impaired People

Project Work Phase-2 · Visvesvaraya Technological University, Belagavi Department of Computer Science and Engineering, Angadi Institute of Technology and Management, Belagavi · 2025–2026

Submitted by	Ms. Aishwarya Kumar Telasinge (2AG23CS007) · Ms. Anuja Sutar (2AG23CS016) · Ms. Aishwarya Channashetti (2AG23CS006) · Ms. Chandrika Bannimatti (2AG23CS032)
Guide	Prof. Avanti Patil, Assistant Professor, Dept. of CSE, AITM Belagavi
Scope of this document	The methods by which the system was designed, implemented and validated

CHAPTER 1

Methodological Overview

1.1 Nature of the approach

This project is **constructive (design-and-build) research**, not an experimental or survey study. The contribution is a working artefact — a real-time indoor navigation system — and the methodology is therefore the disciplined process by which that artefact was specified, decomposed, implemented and verified.

The work follows an **incremental, verification-driven development method.** Each subsystem was built to a written requirement, given an executable test where its behaviour could be expressed as a pure function of its inputs, and integrated only once it behaved correctly in isolation. No subsystem was considered complete on the strength of a visual demonstration alone.

1.2 The central methodological decision: separation of sensing roles

The single decision that shapes every method described in this document is the **deliberate division of labour between the two sensing modalities:**

MODALITY	QUESTION IT ANSWERS	QUESTION IT IS *NOT* ASKED
Surveillance camera	*Where is the user, and what is on the floor around them?*	Who is that person?
BLE beacon	*Is the registered user present, and roughly how far away?*	Where exactly is the user?

This split is a methodological constraint, not an implementation detail. With no BLE sniffer array deployed, BLE cannot triangulate; attempting to derive position from a single RSSI reading would introduce metre-scale error into the one quantity — distance to an obstacle — on which user safety depends. The method therefore assigns **all metric localisation to the camera**, whose homography based measurement is geometrically well-posed, and restricts BLE to the **identity and presence** question that vision alone cannot answer.

1.3 Governing design principles

Five principles were applied consistently throughout, and each is traceable to specific structural choices in the delivered code:

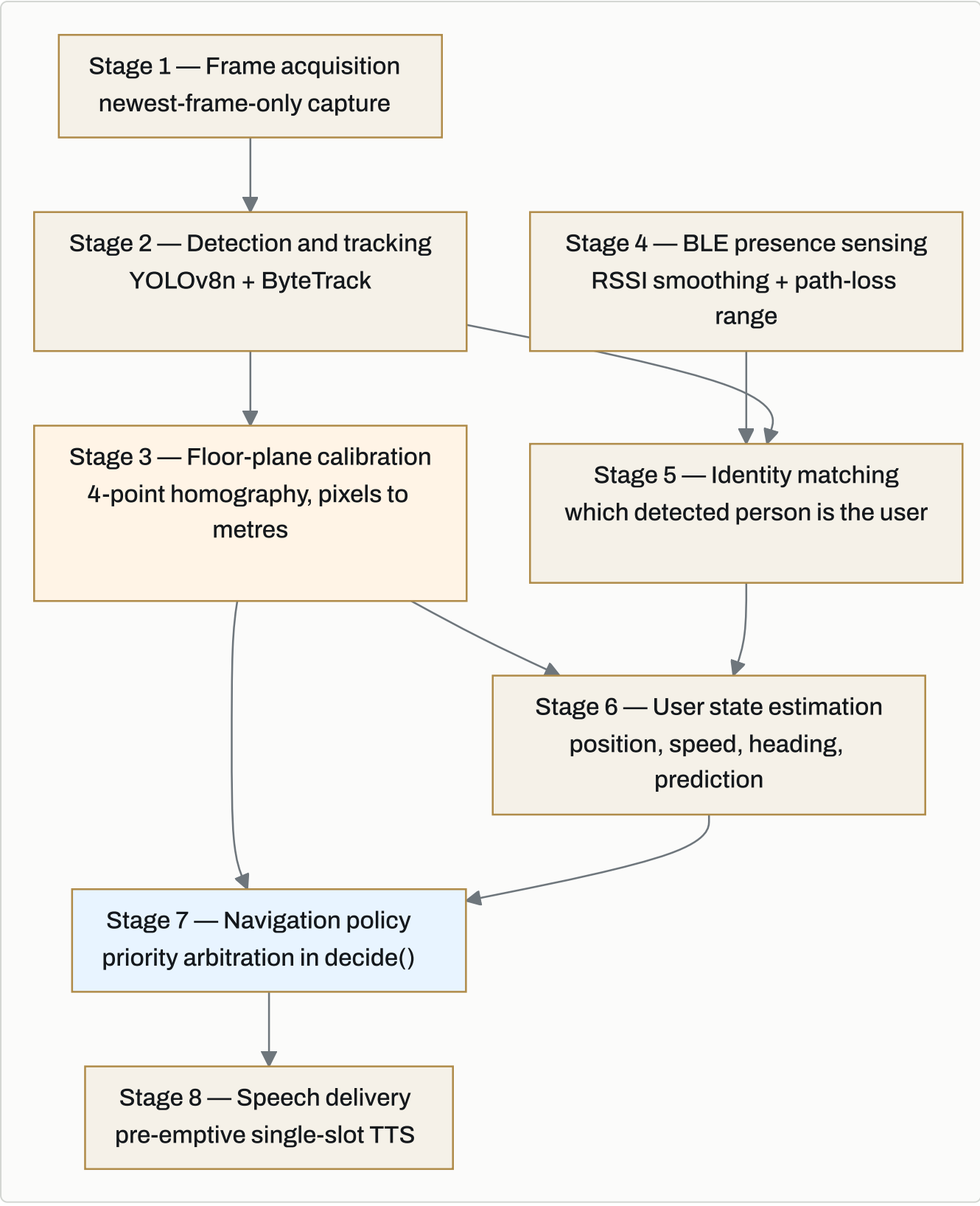
#	PRINCIPLE	HOW IT WAS ENFORCED
P1	Separate policy from mechanism	All navigation rules live in a single pure function, `decide()`, containing no threads, no clock and no hardware access
P2	Decouple component rates	Capture, detection, BLE scanning and decision-making each run at their own rate in their own thread
P3	Degrade, never crash	Every external dependency has a defined failure behaviour that preserves partial function
P4	No magic numbers	Every threshold resides in `config/config.yaml`; no tunable constant is written into logic
P5	Testability by construction	Geometry and policy were written as pure functions *so that* they could be tested without hardware

---

CHAPTER 2

System Methodology — The Processing Pipeline

The system transforms raw sensor input into a spoken instruction through eight sequential stages. Sections 3–10 describe the method used at each stage.



Method note. Stage 3 is drawn feeding both Stage 6 and Stage 7 because the homography is the *metric foundation* of the entire system: every distance the system speaks is ultimately derived from it. This is why calibration accuracy was treated as a first-order concern (Section 5.4) rather than a setup chore.

Stage 1 — Frame Acquisition Method

3.1 Problem

A camera produces frames at approximately 30 Hz. On the target hardware (an Intel i3 class demo machine) the detector consumes them at 4–20 Hz. Any queue between the two grows without bound, and the system ends up reasoning about where the user *was* several seconds ago — the most dangerous possible failure for a safety system.

3.2 Method adopted — newest-frame-only capture

A dedicated capture thread continuously reads from the camera and **overwrites** a single-slot buffer rather than appending to a queue. Frames that the detector was too slow to consume are discarded rather than accumulated.

This trades throughput for **latency bounded by one frame interval**. The justification is that a navigation system derives no value from a stale frame: an obstacle warning computed from a two-second-old image is not merely late, it is wrong.

3.3 Rationale for rejecting the alternative

A bounded FIFO queue was considered and rejected. A queue of depth n admits up to n frame intervals of latency, and choosing n correctly requires knowing the detector's speed in advance — which varies with machine, image size and scene complexity. The single-slot method is self-regulating and requires no tuning.

CHAPTER 4

Stage 2 — Detection and Tracking Method

4.1 Model selection method

YOLOv8n was selected by applying the following criteria in order of priority:

CRITERION	REQUIREMENT	WHY IT DOMINATED
CPU inference speed	≥ 10 FPS on commodity CPU	The demo hardware has no usable GPU
Single-pass multi-class output	Persons *and* obstacles from one forward pass	Running two models would double the per-frame cost
Pre-trained indoor class coverage	Chairs, tables, couches, bags, etc.	Avoids the cost of collecting and annotating a bespoke dataset
Integrated tracking	Identity persistence across frames	Required by the matching method (Section 7)

The nano variant was chosen over larger variants specifically because the guidance loop's timing requirement (Section 9) is better served by frequent approximate detections than by infrequent precise ones.

4.2 Class partitioning method

Rather than train a bespoke model, the method **partitions the pre-trained COCO output** by class id at inference time:

- **Person** — COCO class `0`, forwarded to the tracker and the matcher.
- **Obstacles** — a curated subset of 17 COCO classes chosen for their relevance to indoor walkable space: `bicycle`, `bench`, `backpack`, `handbag`, `suitcase`, `bottle`, `cup`, `chair`, `couch`, `potted plant`, `bed`, `dining table`, `tv`, `laptop`, `keyboard`, `book`, `vase`.
- **All other classes** — discarded.

Curating the subset rather than treating every detection as an obstacle prevents the policy from issuing warnings about wall-mounted or irrelevant objects that pose no trip hazard.

4.3 Ground-contact estimation method

Every distance in the system depends on mapping an object to a floor coordinate, and **only one pixel of a bounding box lies on the floor plane**: the bottom-centre point where the object meets the ground.

```
foot_point = ( (x1 + x2) / 2 ,  y2 )
```

The box centroid is explicitly *not* used. A centroid sits at torso height for a person and mid-body for furniture; projecting it through a floor homography would place the object systematically further from the camera than it truly is, inflating every distance estimate. This single choice is the difference between metric estimates that are usable and estimates that are biased.

4.4 Obstacle persistence method — the TTL cache

Static obstacles are intermittently missed by the detector, particularly when partially occluded by the walking user. A naive method would delete an obstacle the moment it went undetected, causing warnings to flicker on and off.

The method adopted maintains a **time-to-live cache**: a detected obstacle remains on the internal map for `obstacle_ttl_s` (2.5 s) after it was last seen, and is refreshed on each re-detection. The assumption is explicit and stated as a limitation — furniture is treated as static over a 2.5 s horizon, which holds for chairs and tables but not for a person walking through the frame. Persons are therefore excluded from this cache and handled by the tracker instead.

CHAPTER 5

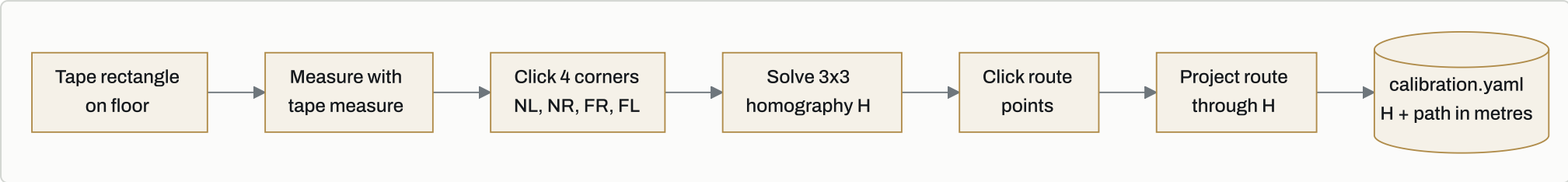
Stage 3 — Floor-Plane Calibration Method

5.1 Problem

A camera reports positions in pixels. Navigation requires metres. The mapping between the two depends on the camera's height, tilt and lens, none of which are known a priori.

5.2 Method adopted — planar homography from a measured rectangle

The method exploits a geometric fact: **all pedestrian navigation happens on a single plane, the floor.** The projective mapping between one plane and another is a homography, fully determined by four point correspondences. It requires no camera intrinsics, no lens calibration and no depth sensor.



The operator performs a one-time procedure:

1. Tape a rectangle on the floor within the camera's view.
2. **Measure it with a tape measure** and supply the true width and depth in metres.
3. Freeze a frame and click the four corners in a fixed order: near-left, near-right, far-right, far-left.
4. Click a sequence of points defining the safe walking route.
5. Save.

The four clicked pixel coordinates are paired with their known metric coordinates `(0,0), (w,0), (w,d), (0,d)` and solved for the 3×3 homography **H**. The route points are then projected through **H** and stored in metres, so the safe path is defined in world coordinates independent of the image.

Thereafter, any pixel is converted to floor metres by `(x, y) = H · (px, py, 1)` in homogeneous coordinates, and the inverse H^{-1} renders metric overlays back onto the image.

5.3 Camera placement method

Mounting geometry was specified as part of the method, not left to the installer:

PARAMETER	SPECIFIED VALUE	METHODOLOGICAL JUSTIFICATION
Height	1.8–2.2 m	Below this the user's body occludes the floor behind them
Downward tilt	25–35°	Sets the depth resolution of the homography
Post-calibration movement	None permitted	H encodes the exact pose; any movement silently invalidates every distance

At table height the perspective compresses: many metres of floor map to very few pixels in the far field, and the homography amplifies pixel noise into large metric error. The tilt specification exists to keep this conditioning acceptable.

5.4 Error-propagation reasoning

Because the tape-measured rectangle defines the metric scale of the entire system, **measurement error at calibration time propagates proportionally into every distance the system ever speaks.** A 10 % error in the measured rectangle width yields approximately 10 % error in every subsequent obstacle distance — enough to convert a 1.5 m stop warning into a 1.65 m warning. Calibration was therefore treated methodologically as the accuracy-determining step, and the procedure mandates a physical tape measure rather than an estimate.

CHAPTER 6

Stage 4 — BLE Presence Sensing Method

6.1 Beacon method

An **ESP32** module is programmed as a BLE advertiser broadcasting under the identifier `BLENAV_BEACON` at elevated transmit power (+9 dBm) to maximise range and stabilise RSSI. The user

carries this beacon; no smartphone application is required, and no wearable camera or sensor is worn.

6.2 Scanning and identification method

A dedicated thread scans at 1 Hz. Target identification uses a **two-tier method**:

1. If a MAC address is configured, match on address — exact and unambiguous.
2. Otherwise, match on advertised local name containing `BLENAV_BEACON`.

The name-based fallback is a deliberate deployability decision: the system is operable immediately after flashing the beacon, without first reading its MAC from a serial monitor and editing configuration.

6.3 RSSI conditioning method

Raw RSSI is unstable, fluctuating several dBm between consecutive readings from multipath and body shadowing alone. Two conditioning steps are applied:

Moving average. RSSI is smoothed over a 5-sample window before any decision uses it.

Hysteresis on presence. The user is considered present when smoothed RSSI exceeds `present_rssi` (−85 dBm), and is only declared absent after `lost_after_s` (5 s) without a qualifying reading. This asymmetry prevents a single dropped advertisement from revoking the target lock mid-instruction.

6.4 Coarse ranging method and its stated limitation

Distance is estimated with the **log-distance path-loss model**:

```
d = 10 ^ ( (txPower - RSSI) / (10 · n) )
```

where `txPower` (−59 dBm) is the calibrated RSSI at one metre and `n` (2.5) the environmental path loss exponent for indoor space.

This estimate is used for **exactly one purpose**: as a weak tie-breaking prior when several people are in frame (Section 7). It is deliberately never used to compute an obstacle distance or to position the user. The method acknowledges that indoor RSSI ranging carries metre-scale error, and confines that error to a decision where it can only affect *which* person is guided, never *what distance* is reported.

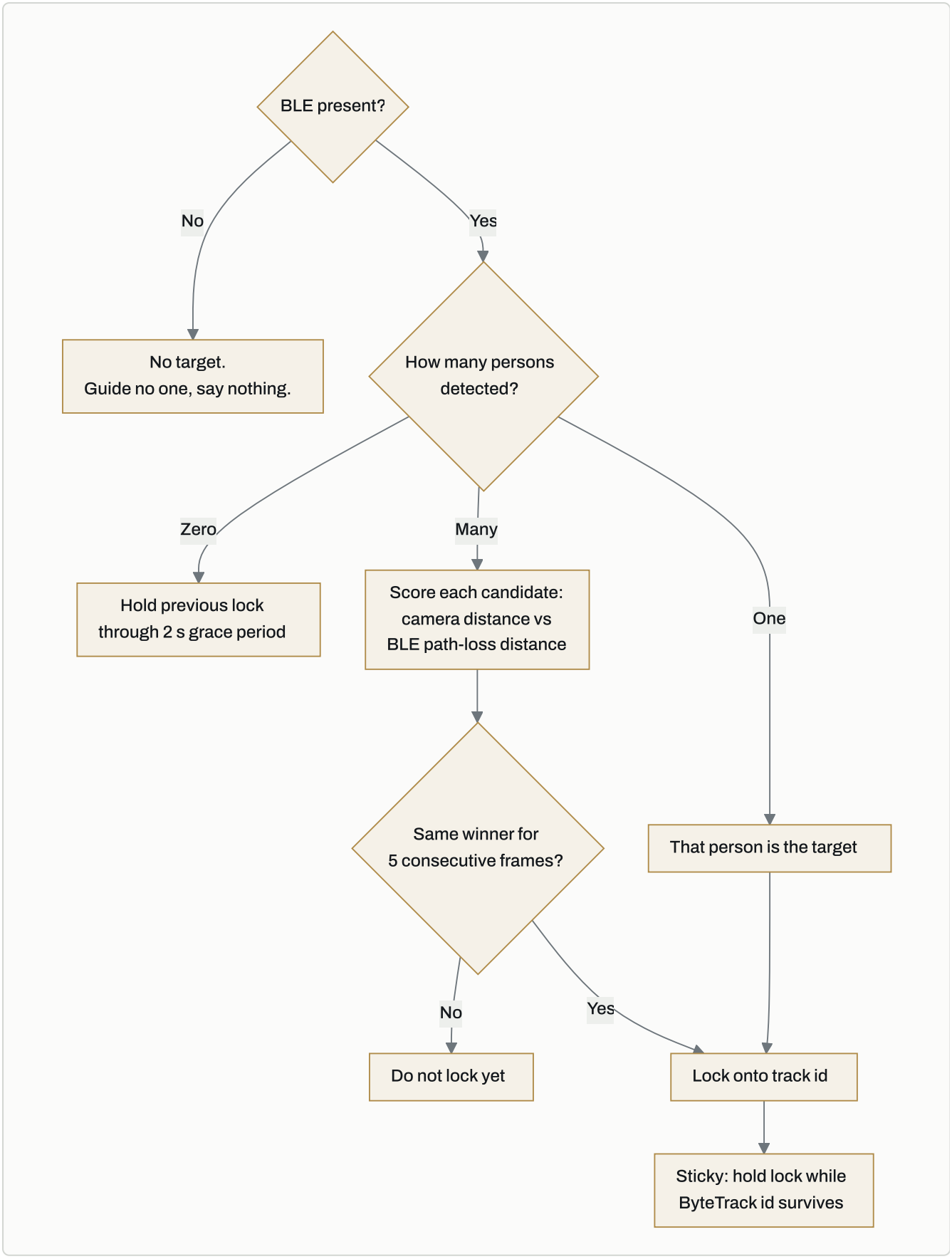
CHAPTER 7

Stage 5 — Identity Matching Method

7.1 Problem

The camera sees *people*. It cannot see *which* person is the registered user. BLE knows the user is present but not where. Matching must fuse the two.

7.2 Case-based matching method



7.3 Three safeguards against mis-identification

The method's structure is dominated by the consequence of a wrong answer — guiding a stranger, or worse, guiding the user using a stranger's position. Three mechanisms guard against this:

MECHANISM	METHOD	FAILURE IT PREVENTS
Stability streak	The same candidate must win 5 consecutive frames before a lock is granted	A single noisy RSSI sample capturing the lock
Sticky lock	Once locked, the lock persists as long as the tracker keeps that id alive; scoring is not re-run	Guidance oscillating between two nearby people
Occlusion grace	If the track disappears, the lock is held for 2 s before release	A brief occlusion causing target loss and re-acquisition

7.4 Justification of the single-person case

That one detected person is accepted as the target without scoring is a **correct inference, not a convenience**: BLE has established that the registered user is within range, and the camera reports exactly one person in the calibrated area. No alternative hypothesis exists. The case is called out explicitly because it is also the demonstration case, and its correctness should not be mistaken for a shortcut.

CHAPTER 8

Stage 6 — User State Estimation Method

8.1 Heading estimation

Guidance phrased as *"move left"* is meaningless without knowing which way the user faces. No compass or IMU is worn, so heading is derived **from motion**: the displacement vector across a 0.8 s history window, normalised.

The window length is itself a methodological compromise — short enough to track turns responsively, long enough that per-frame detection jitter does not rotate the estimated heading spuriously.

8.2 Fallback when the user is stationary

A stationary user has no motion and therefore no derivable heading. Rather than refuse to guide, the method **substitutes the direction of the predefined route** at the user's current position. On a known route this is a sound assumption and strictly better than silence.

If neither motion nor route is available, the system reports honestly that it has no heading yet and stays silent, rather than guessing.

8.3 Egocentric transformation method

Obstacles are detected in world coordinates but must be described relative to the user. Given user position $\mathbf{p}$, unit heading $\mathbf{h}$ and obstacle position $\mathbf{o}$, with $\mathbf{v} = \mathbf{o} - \mathbf{p}$:

```
forward_m = v · h            (dot product — distance ahead along heading)
lateral_m = h × v            (cross product z — positive means to the user's LEFT)
```

This yields the two quantities that instruction generation actually needs, in the user's own frame of reference.

8.4 Corridor filtering method

Not every detected obstacle is a threat. A chair three metres off to one side is irrelevant, and announcing it would erode trust in the system's warnings. An obstacle is admitted only if it lies within the **walking corridor**:

```
0 < forward_m ≤ max_forward_m     and     |lateral_m| ≤ half_width_m
```

with `half_width_m` = 0.55 m, representing shoulder width plus a clearance margin. Stairs are tested against a corridor of double width, on the reasoning that a fall hazard warrants warning even when it is not squarely ahead.

8.5 Extrapolation between detector passes

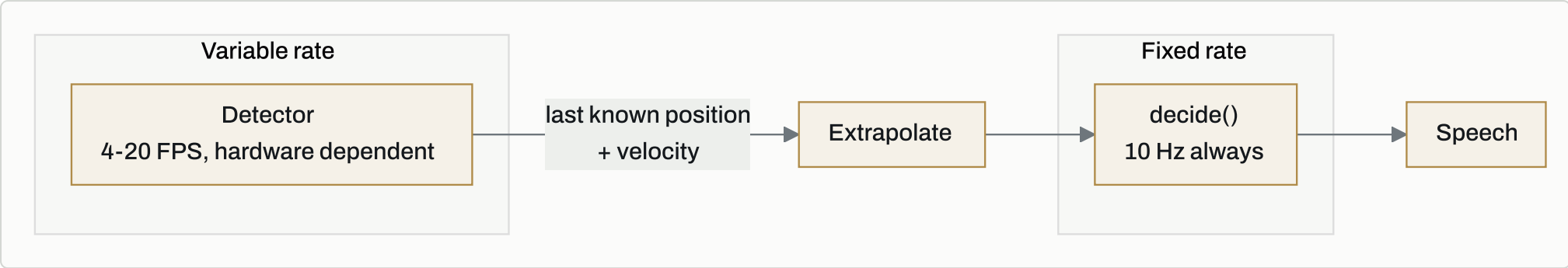
Because the detector may run several times slower than the decision loop, the user's position is **predicted from last known position and velocity** for decisions that fall between detector passes. This is what allows warnings to remain timely on slow hardware (Section 9).

CHAPTER 9

Stage 7 — Navigation Policy Method

9.1 Fixed-rate decision method

The guidance loop runs at a **fixed 10 Hz regardless of detector throughput.** This is the methodological answer to slow hardware: rather than allowing instruction timing to degrade with inference speed, the decision rate is held constant and the position input is extrapolated (Section 8.5).



At 4 FPS a purely reactive system would decide once every 250 ms; a user walking at 1.2 m/s covers 30 cm in that interval. Fixed-rate decision-making with extrapolation reduces the decision interval to 100 ms, restoring the timing margin needed for a stop warning to be actionable.

9.2 Priority arbitration method

At any instant several conditions may hold simultaneously — an obstacle ahead *and* a drift off route. Speech is strictly serial, so the policy must select exactly one instruction. The method is **strict priority arbitration**: conditions are evaluated in a fixed order and the first match returns immediately.

PRIORITY	CONDITION	INSTRUCTION	COOLDOWN
P0	Obstacle within 1.5 m in the walking corridor	*"Stop. Obstacle ahead."*	2.0 s
P1	Stairs within 4.0 m ahead	*"Caution. Stairs ahead."*	4.0 s
P2	Obstacle within 3.5 m, room to steer	*"Obstacle ahead. Move left / right."*	3.0 s
P3	More than 0.6 m off the predefined route	*"You are drifting left. Move right."*	3.0 s
P4	Path clear	*"Continue straight."*	6.0 s
P5	Not ready — no BLE, no lock, uncalibrated	*(silence)*	10–30 s

Ordering justification. Stairs outrank ordinary obstacles (P1 above P2) because the consequence of walking into a chair is a collision, while the consequence of walking into a descending staircase is a fall. Ranking by *severity of outcome* rather than by proximity is a deliberate safety choice.

9.3 Avoidance-direction method

When steering room exists, the policy must choose a side. Obstacles in the corridor are counted by side and the **less obstructed side** is chosen. On an exact tie, the direction *away from the nearest single obstacle* is selected. On a remaining tie the system keeps right, following the prevailing pedestrian convention — a defined, predictable behaviour rather than an arbitrary one.

9.4 Silence as a designed output

Priority P5 emits **no speech at all**. When BLE is absent, the target is not locked, or the system is uncalibrated, the correct output is silence rather than a guess. A navigation aid that speaks when it does not know what it is looking at is worse than one that says nothing, because the user cannot distinguish a confident instruction from a fabricated one.

9.5 Purity as a methodological device

`decide()` is written as a **pure function**: its output depends only on its arguments, and it performs no I/O, consults no clock and touches no hardware. The threaded loop that gathers inputs contains no navigation logic whatsoever.

This separation was adopted specifically to make the policy testable. Because `decide()` is pure, any navigation scenario — a user 1.2 m from a chair, drifting 0.8 m left, with stairs 3 m ahead — can be

constructed as a data structure and asserted against, with no camera, no beacon and no room. Section 11 depends entirely on this property.

CHAPTER 10

Stage 8 — Speech Delivery Method

10.1 Problem

Text-to-speech is inherently serial and an utterance takes 1–2 s. If a *"Continue straight"* is in progress when an obstacle appears, the urgent warning must not wait behind it.

10.2 Single-slot pre-emptive queueing method

A single pending slot is maintained rather than a queue:

- A new instruction of **higher priority overwrites** a pending lower-priority instruction.
- A new instruction of **equal or lower priority is discarded** while a pending one waits.
- P0 emergency warnings may **bypass** cooldown and priority entirely.

A conventional FIFO queue was rejected: it would deliver every instruction eventually, including stale ones describing a situation that had already passed. Discarding superseded instructions is correct behaviour for a real-time system.

10.3 Repetition suppression method

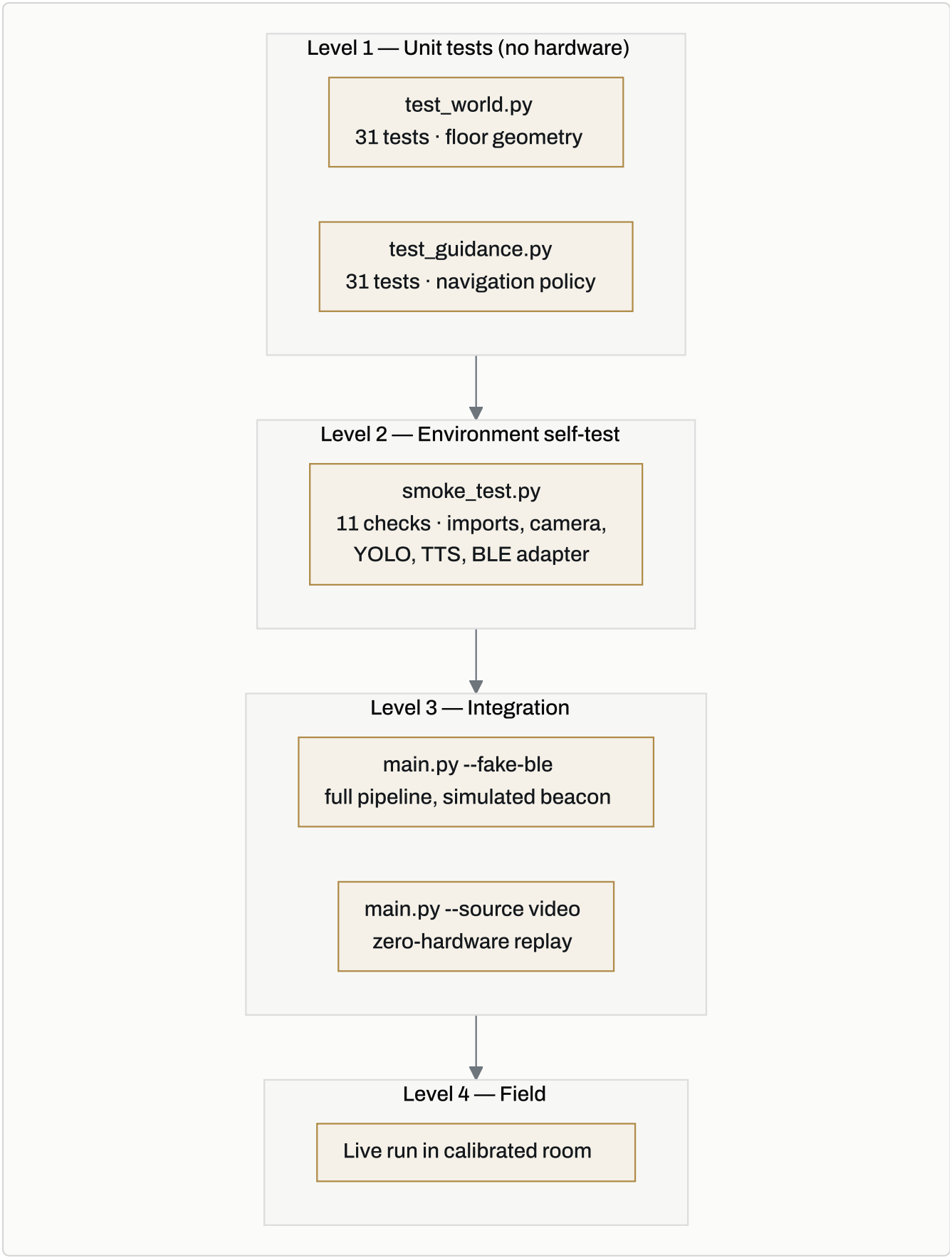
Each distinct phrase carries a **cooldown** (see the table in Section 9.2) during which an identical phrase is suppressed. Cooldowns are graded by urgency: *"Stop. Obstacle ahead."* may repeat every 2 s because the hazard is immediate, whereas *"Continue straight."* is limited to once per 6 s because constant reassurance becomes noise the user learns to ignore.

CHAPTER 11

Validation Methodology

11.1 Strategy

Validation was designed around a constraint discovered early: a test that requires a camera, a beacon and a calibrated room can only be run in that room, which means in practice it is rarely run. The strategy therefore **maximises the proportion of the system testable without hardware**, and this requirement drove the architecture rather than following from it.



11.2 Level 1 — Pure-function unit testing

The two components carrying the highest correctness risk — floor geometry and navigation policy — were written as pure functions expressly so they could be tested exhaustively without hardware.

- `test_world.py` **(31 tests)** — homography round-trips, route deviation and side determination, egocentric transformation, corridor admission and rejection, free-side selection including tie cases.
- `test_guidance.py` **(31 tests)** — every priority level and, critically, the **boundaries and orderings between them**: that stairs outrank a more distant obstacle, that an obstacle just inside 1.5 m produces a stop while one just outside produces an avoid, that cooldowns suppress repetition at the correct rate.

Testing the transitions rather than the states is deliberate: the defects that matter in a priority policy occur at its boundaries.

11.3 Level 2 — Environment self-test

`smoke_test.py` performs 11 checks in five groups: library imports, camera capture with measured FPS, YOLO inference latency at two image sizes, text-to-speech driver availability, and a live BLE scan listing visible devices with RSSI.

Its methodological purpose is **diagnostic isolation**. On an unfamiliar machine it separates environment faults from system faults before any debugging of the application begins, and it is specified as the first command to run on any new deployment.

11.4 Levels 3 and 4 — Integration and field validation

Integration testing uses two substitution mechanisms that allow the complete pipeline to be exercised with progressively less hardware:

MODE	SUBSTITUTION	PURPOSE
`--fake-ble`	Simulated always-present beacon at fixed RSSI	Validate the full pipeline before the ESP32 is flashed
`--source video.mp4`	Recorded video replaces the live camera	Reproducible replay with no hardware at all
`--mute`	Captions replace speech	Verify detection and overlays without audio

Field validation is performed in the calibrated room with all hardware live, following a defined sequence: user enters → BLE detected → camera detects person → target locked → user walks → route deviation checked → obstacle warning issued → user guided around.

11.5 Recorded validation results

VALIDATION LEVEL	RESULT
`test_world.py`	31 / 31 passed
`test_guidance.py`	31 / 31 passed
`smoke_test.py`	11 / 11 passed
Detector latency @ 320 px	26 – 38 ms per frame
End-to-end pipeline, uncalibrated	Runs; detects and tracks; correctly withholds metric guidance

VALIDATION LEVEL	RESULT
End-to-end pipeline, calibrated	Runs; emits metric guidance decisions at the specified rate

The measured detector latency confirms the timing premise of Section 9.1: inference at 26–38 ms supports the 100 ms decision interval with margin on the target hardware.

CHAPTER 12

Degradation Methodology

Because the system is a safety aid, its behaviour under partial failure was specified as a design requirement rather than discovered after the fact. Each external dependency has a defined degradation path that preserves as much function as remains sound:

FAILURE	METHOD	RETAINED FUNCTION
No calibration file	Load returns null; policy short-circuits to P5	Detection and tracking continue; HUD states *NOT CALIBRATED*; **no metric guidance is fabricated**
BLE adapter missing	Scanner retries every 3 s; `--fake-ble` available	System runs; guidance withheld until presence is established
TTS unavailable	Speaker degrades to text output	Instructions still printed and rendered on screen
Frames dropped	Extrapolate from last known velocity	Guidance continues at full 10 Hz
User briefly occluded	Target lock held through 2 s grace	Lock survives; no spurious re-acquisition
Stairs model absent	Stairs list stays empty; P1 never fires	All other priorities unaffected

The governing rule is visible in the first and last rows: **when the system cannot compute a quantity soundly, it declines to speak rather than substituting an approximation.** An uncalibrated system does not guess at metres.

CHAPTER 13

Tools, Materials and Configuration Method

13.1 Implementation stack

COMPONENT	SELECTION	BASIS FOR SELECTION
Language	Python 3.11	Ecosystem maturity for CV; adequate given per-frame cost is dominated by native inference
Detection	Ultralytics YOLOv8n	CPU-viable, single-pass multi-class, integrated tracking
Tracking	ByteTrack	Bundled with the detector; retains identity through low-confidence frames

COMPONENT	SELECTION	BASIS FOR SELECTION
Vision / geometry	OpenCV	Capture, homography solving, overlay rendering
BLE	`bleak`	Cross-platform asynchronous BLE scanning
Speech	`pyttsx3`	Fully offline; no network dependency at runtime
Beacon firmware	Arduino C++ on ESP32	Low-cost, adequate transmit power
Configuration	YAML	Human-editable without code changes

13.2 Configuration method

Principle **P4** is enforced structurally: **every** tunable threshold resides in `config/config.yaml` — detection confidence and image size, all five guidance distance thresholds, loop rate, all cooldowns, BLE path-loss parameters, RSSI window and presence threshold, speech rate and volume.

The methodological consequence is that the system can be re-tuned for a different room, a different walking speed or a different machine **without modifying or re-testing a single line of logic**. Calibration data is likewise externalised to `calibration.yaml`, keeping room-specific geometry fully separate from both code and configuration.

CHAPTER 14

Methodological Limitations

Stated explicitly, as required of an honest method:

1. **Infrastructure-bound coverage.** The system operates only within one calibrated camera view. Extending coverage requires a per-camera homography and a hand-off rule at coverage boundaries.
2. **Static-obstacle assumption.** The TTL cache assumes obstacles do not move within 2.5 s. Valid for furniture; not valid for other pedestrians, which is why persons are excluded from it.
3. **Coarse BLE ranging.** Path-loss distance carries metre-scale error indoors. The method confines this error to candidate tie-breaking and never to a reported distance.
4. **Motion-derived heading.** A stationary user off the defined route yields no heading. The route fallback covers the on-route case; the residual case is answered with silence.
5. **Calibration sensitivity.** Metric accuracy is bounded by tape-measurement accuracy and is silently invalidated if the camera is moved after calibration.
6. **Stairs detection pending.** The P1 policy, thresholds and tests are implemented and passing, but a fine-tuned stairs model has not yet been trained; the priority level is currently unreachable at runtime.
7. **No end-user trials.** Validation to date is technical. Trials with visually impaired participants — measuring instruction comprehension, reaction time and perceived safety — are Phase-3 work.

CHAPTER 15

Summary

The methodology is characterised by four decisions, each traceable to a specific structural property of the delivered system:

1. **Sensing roles were separated by geometric well-posedness.** Vision localises because homography-based measurement is well-conditioned; BLE identifies because it is the only modality that can. Neither is asked the question it would answer badly.

1. **Component rates were decoupled and the decision rate fixed.** Guidance decides at a constant 10 Hz with extrapolated position, so instruction timing does not degrade with inference speed — the property that makes the system viable on commodity hardware.

1. **Policy was isolated as a pure function.** Every navigation rule lives in `decide()`, which touches no hardware and no clock. This made 62 hardware-free tests possible and confined all safety-critical logic to one auditable, deterministic place.

1. **Uncertainty was answered with silence.** At every stage where a quantity could not be computed soundly — uncalibrated, unlocked, no heading — the specified output is no output. For an aid a user must trust while walking, a withheld instruction is safer than a fabricated one.